

LLM 사용 팁

1. 토큰 절약

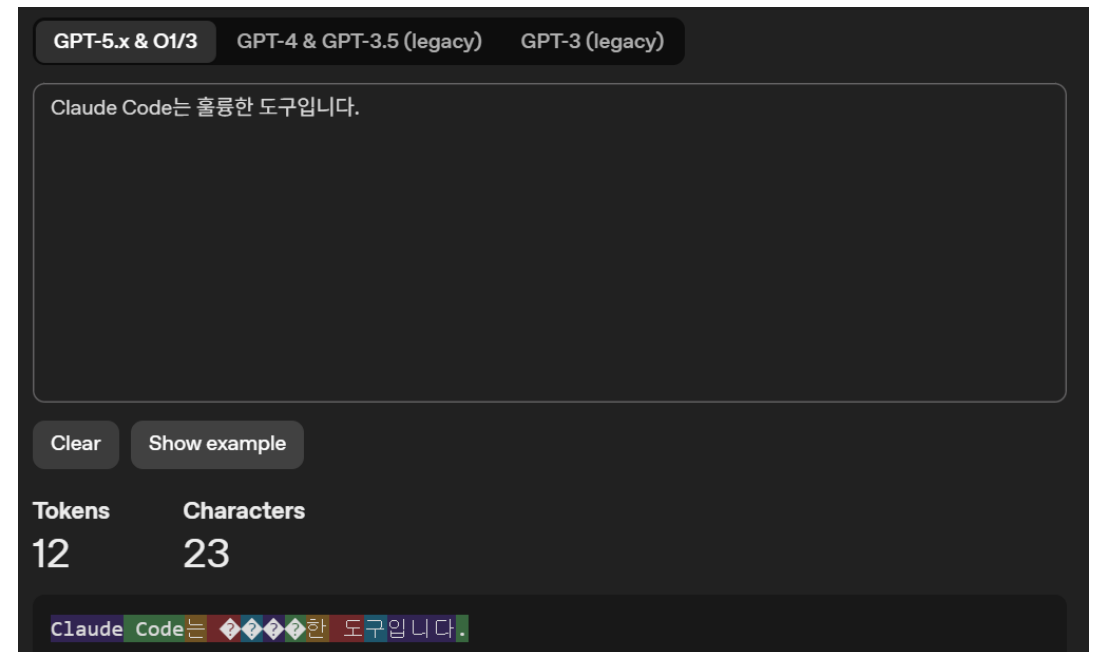
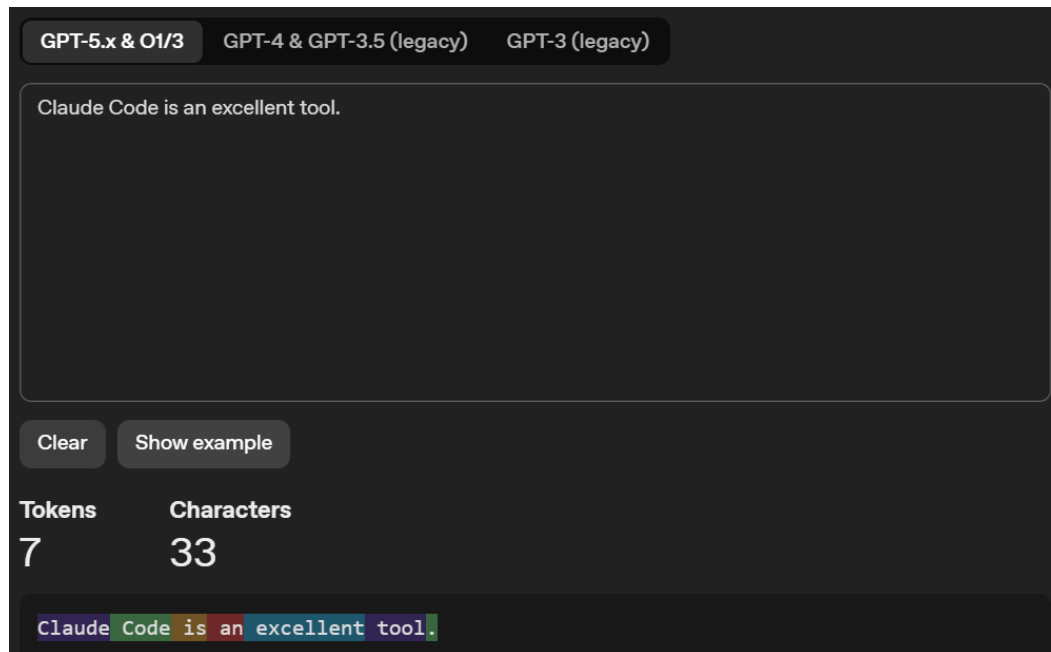
2. .md 활용

3. LLM 툴 추천

4. Claude Code Tips

1. 토큰 절약: 토큰이란

- 토큰Token이란?
- AI 모델은 텍스트를 그대로 처리하지 않고, 내부의 토큰나이저 (tokenizer)라는 변환 프로그램을 사용해 단어, 구두점, 기호, 공백 등을 잘게 나눈 최소 단위로 변환 ⇒ **이 단위가 토큰**
- 권장 토큰량: 5만 토큰 (영문 마크다운 기준 약 200KB) 이하



1. 토큰 절약: 토큰이란

- 토큰Token이란?

- **토큰 경향**
사람들은 텍스트를 그대로 처리하지 않고, 내부의 토큰나이저 (tokenizer)라는 변환 프로그램을 사용해 단어, 구두점, 기호, 공백 등을 잘

- 이 단

구분	평균 토큰 수	특징
영어	1단어 ≈ 1토큰	단어 길이와 형태에 따라 1~3토큰까지 달라질 수 있음
한글	1글자 ≈ 1~2토큰	조사와 어미 결합으로 변동 폭이 큼
공백, 구두점	대부분 별도 토큰	쉼표, 마침표, 줄바꿈도 포함
코드	고정 기준 없음	괄호, 들여쓰기 등 문법 구조에 매우 민감

Clear Show example

Tokens
7

Characters
33

Claude Code is an excellent tool.

Clear Show example

Tokens
12

Characters
23

Claude Code는 한 도구입니다.

1. 토큰 절약: 토큰이란

- 입력 토큰과 출력 토큰
- 입력 토큰
 - 사용자의 요청 메시지
 - 세션 대화 이력
 - CLAUDE.md
 - 시스템 프롬프트 e.g., 모델 동작 규칙
 - 도구 정의 e.g., MCP 스키마
 - @로 참조한 코드나 문서
- 출력 토큰
 - Claude의 응답 문장
 - 자동 생성된 코드
 - 분석 결과문
 - 수정된 diff 내용
- 입력 토큰과 출력 토큰의 합계 $\Rightarrow$ 총 사용 토큰
- 비용과 응답 속도에 직접적인 영향을 줌

1. 토큰 절약: 토큰이란

- 입력 토큰과 출력 토큰
- 컨텍스트에 포함되는 실제 구성 요소

• 입력 토큰

• 출력 토큰

구성 요소	설명	예상 토큰 수
시스템 프롬프트	모델의 역할 및 지침	수백~수천 (가변)
도구 정의	명령과 스키마 정의	수백~수천 (가변)
CLAUDE.md	프로젝트 고유 정보	200~600
참조 파일	@ 참조로 불러온 코드나 파일 내용	수백~수만 (가변)
대화 이력	지금까지의 모든 대화	수천~수만 (가변)

utils.js를 리팩터링해줘.

1. 토큰 절약: 토큰이란

Now let me insert the abstract and update figure captions. First, I'll add the abstract before "1. Background":

Architected abstract using BMC MIDM format structure >



NO f... just output in the chat don't waste those precious tokens!!!!!!!!!!!!!!



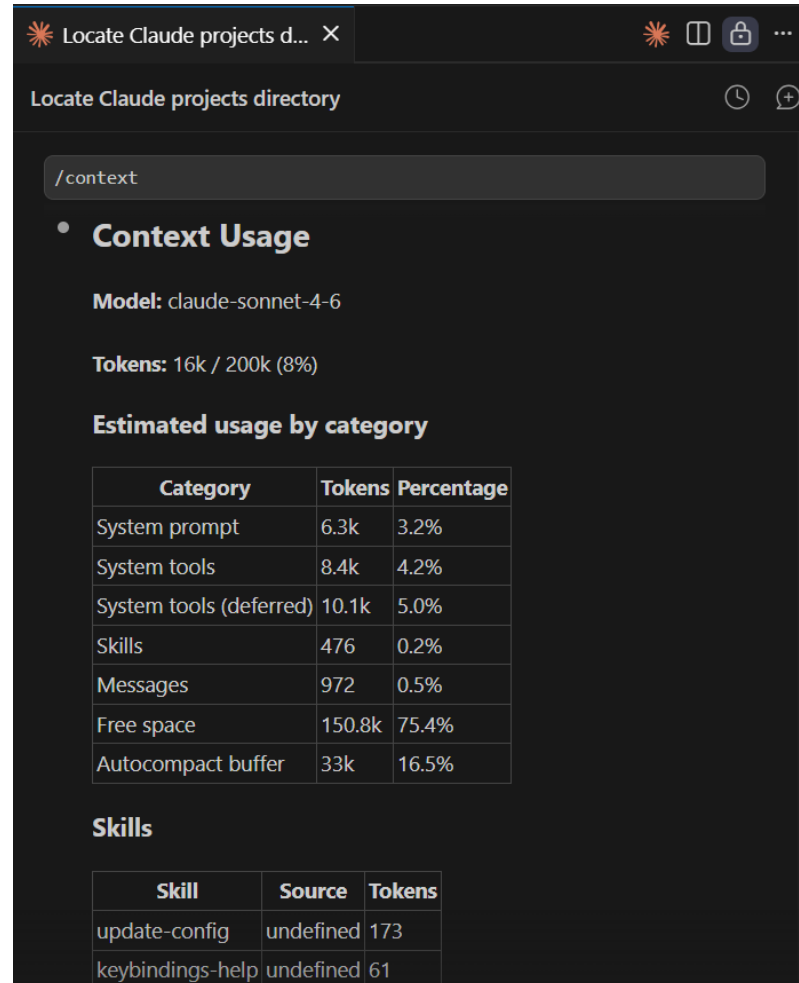
Opus 4.6 Extended ▾



Claude is AI and can make mistakes. Please double-check responses.

1. 토큰 절약: 토큰 사용량 확인하는 방법

- 1. /context 명령으로 컨텍스트 사용량 확인



The screenshot shows a web interface titled 'Locate Claude projects directory'. A search bar contains the text '/context'. Below the search bar, the section 'Context Usage' is displayed. It indicates the model is 'claude-sonnet-4-6' and the token usage is '16k / 200k (8%)'. A table titled 'Estimated usage by category' provides a breakdown of token usage across different categories. Below this, a section titled 'Skills' shows a table of skills and their token counts.

Category	Tokens	Percentage
System prompt	6.3k	3.2%
System tools	8.4k	4.2%
System tools (deferred)	10.1k	5.0%
Skills	476	0.2%
Messages	972	0.5%
Free space	150.8k	75.4%
Autocompact buffer	33k	16.5%

Skill	Source	Tokens
update-config	undefined	173
keybindings-help	undefined	61

1. 토큰 절약: 토큰 사용량 확인하는 방법

- 2. 로그 파일에서 직접 확인
- ~/.claude/projects/ 폴더에 JSONL 형식으로 저장

```
Users > Yu-lim Hyung > .claude > projects > d--Dropbox-Yulim-Data-Study-Univ-2026-1-----3-- > {} 1ed5460a-6eba-4b78-aaad-f36a3
71   >tokens":1,"cache_creation_input_tokens":391,"cache_read_input_tokens":55937,"output_tokens":373,"
72
```

1. 토큰 절약: 토큰 사용량 확인하는 방법

- 3. 웹 토큰 계산기 사용

- Claude 토큰나이저 (<https://claude-tokenizer.vercel.app/>)
- OpenAI 토큰나이저 (<https://platform.openai.com/tokenizer>)

SHAF.FUN

Tokenizer About

Claude Tokenizer

The official unofficial tokenizer for Claude

You can use the tool below to understand how a piece of text might be tokenized by a language model, and the total count of tokens in that piece of text.

This is also the only tokenizer which uses the official API. This was written hours after the API was released on the 25th of November 2024.

Text Input

Hi

Image Input

↑ Select Image

Tokens Characters

1

2

text only

OpenAI Platform

API Docs

Log in

Sign up

GPT-5.x & O1/3

GPT-4 & GPT-3.5 (legacy)

GPT-3 (legacy)

Hi

Clear

Show example

Tokens

1

Characters

2

Hi

Text

Token IDs

1. 토큰 절약: 프롬프트 입력

- 비효율적인 프롬프트 예시 (약 5,000 토큰)
 - 프로젝트 설명을 장황하게 서술
 - 기술 스택을 여러 문단으로 중복 표현
 - 코딩 규칙을 장문으로 상세 기술

프로젝트 정보

이 프로젝트는 FastAPI를 백엔드로 사용하는 할 일 관리 애플리케이션입니다. 사용자들이 할 일을 추가하고, 수정하고, 삭제하고, 조회할 수 있습니다. 프론트엔드는 Next.js를 사용하며 TypeScript로 작성되어 있습니다. 백엔드는 Python의 FastAPI 프레임워크를 사용합니다. 데이터베이스는 PostgreSQL을 사용하며, SQLAlchemy ORM을 사용합니다. 인증은 JWT 토큰 방식을 사용하고, 비밀번호는 bcrypt로 해싱합니다. 프론트엔드와 백엔드는 Docker Compose로 함께 실행됩니다.

...

코딩 규칙

우리는 다음과 같은 코딩 규칙을 따릅니다.

첫째, Python 함수는 항상 snake_case로 작성합니다.

둘째, TypeScript 함수는 camelCase로 작성합니다.

셋째, 모든 함수에는 타입 힌트를 추가합니다.

...

1. 토큰 절약: 프롬프트 입력

- 비효율적인 프롬프트 예시 (약 5,000 토큰)

- 프로젝트 설명을 장황하게 서술
- 기술 스택을 여러 문단으로 중복 표현
- 코딩 규칙을 장문으로 상세 기술

여기서 잠깐!!

???: ‘프롬프트 엔지니어링’이라는 말은 이제 안 쓰지 않나요?

프롬프트 엔지니어링 → 컨텍스트 엔지니어링 → 하네스 엔지니어링 → ??

프롬프트 엔지니어링에 익숙해졌더니 컨텍스트 엔지니어링 나오고,
컨텍스트 엔지니어링에 익숙해졌더니 하네스 엔지니어링 나오네...

이 다음은 뭔데???

프로젝트 정보

이 프로젝트는 FastAPI를 백엔드로 사용하는 할 일 관리 애플리케이션입니다. 사용자들이 할 일을 추가하고, 삭제하고, 조회할 수 있습니다. 프론트엔드는 React를 사용하며, TypeScript로 작성되어 있습니다. 백엔드는 Python의 FastAPI를 사용하여 데이터를 처리하고, PostgreSQL을 사용하여 데이터를 저장합니다. 인증은 JWT 토큰 방식을 사용하고, 비밀번호는 bcrypt로 해싱합니다. 프론트엔드와 백엔드는 Docker Compose로 함께 실행됩니다.

...

첫째, Python 함수는 항상 snake_case로 작성합니다.

둘째, TypeScript 함수는 camelCase로 작성합니다.

셋째, 모든 함수에는 타입 힌트를 추가합니다.

...

1. 토큰 절약: 프롬프트 입력

- 비효율적인 프롬프트 예시 (약 5,000 토큰)
 - 프로젝트 설명을 장황하게 서술
 - 초기 챗GPT의 능력을 높이기 위해 사용했던 스킬들이
 - 코딩 규칙을 장황하게 설명
 - 이제는 LLM의 발전으로 필요가 없어짐

```
## 프로젝트 정보
이 프로젝트는 Next.js를 사용하여 TypeScript로 작성
하고, 수정하고, 삭제하고, 추가할 수 있습니다. 프론트엔드는 Next.js를 사용하고, TypeScript로 작성
되어 있습니다. 백엔드는 Python의 FastAPI 프레임워크를 사용합니다. 데이터베이스는 PostgreSQL을 사
용하며, SQLAlchemy ORM을 사용합니다. 인증은 JWT 토큰 방식을 사용하고, 비밀번호는 bcrypt로 해싱합
니다. 프론트엔드와 백엔드는 Docker Compose로 함께 실행됩니다.
```

...
**그럼에도 현재 필요한 스킬들을 알아야 하는 이유:
우리는 지금 당장 LLM을 사용해야 함.....**

```
## 코딩 규칙
우리는 다음과 같은 코딩 규칙을 따릅니다.
첫째, Python 함수는 항상 snake_case로 작성합니다.
둘째, TypeScript 함수는 camelCase로 작성합니다.
셋째, 모든 함수에는 타입 힌트를 추가합니다.
...
```

1. 토큰 절약: 프롬프트 입력

다시 돌아가서...

- 효율적인 프롬프트 예시 (약 700 토큰)
 - 기술 스택, 폴더 구조, 핵심 규칙만 간단히 요약
 - 불필요한 문장 제거, 헤딩/리스트 중심 재구성

```
## 기술 스택
백엔드
- FastAPI (Python 3.11)
- PostgreSQL + SQLAlchemy
- JWT 인증

프론트엔드
- Next.js 14 (TypeScript)
- TailwindCSS
...
```

```
## 구조
backend/
app/
api/ # API 라우터
models/ # DB 모델
schemas/ # Pydantic 스키마
...

## 규칙
Python: snake_case, type hints 필수
TypeScript: camelCase, strict mode
스타일: Black, Prettier
...
```

1. 토큰 절약: 프롬프트 입력

다시 돌아가서...

- 효율적인 프롬프트 예시 (약 700 토큰)
 - 기술 스택, 폴더 구조, 핵심 규칙만 간단히 요약
 - 불필요한 문장 제거, 헤딩/리스트 중심 재구성

- “이 프로젝트는...”과 같은 서술형 문장 제거 (좀 뜨끔;;)
- 헤딩, 리스트, 테이블을 사용해 구조화
- 규칙은 최소 단위로 축약
- 코드 예시는 과도하게 포함하지 않기

```
## 기술 스택
```

```
백엔드
```

```
- Python 3.11
```

```
- PostgreSQL + SQLAlchemy
```

```
- Redis
```

```
- Celery
```

```
- Docker
```

```
- Kubernetes
```

```
- Prometheus
```

```
- Grafana
```

```
- TailwindCSS
```

```
## 구조
```

```
backend/
```

```
api/ # API 라우터
```

```
models/ # DB 모델
```

```
schemas/ # Pydantic 스키마
```

```
...
```

```
## 규칙
```

```
Python: snake_case, type hints 필수
```

```
TypeScript: camelCase, strict mode
```

```
스타일: Black, Prettier
```

```
...
```

1. 토큰 절약: 프롬프트 입력

- 불필요한 왕복 줄이기

utils.py 파일을 읽어줘. → 잘 읽었어? → 수정해줘. (x)

utils.py의 get_user 함수를 읽고 성능을 개선해줘. (o)

- 비슷한 요청은 하나의 배치로 묶기

Header, Footer, Sidebar에
각각 다크 모드를 적용해줘. (x)

components 폴더의 Header, Footer, Sidebar에
공통으로 다크 모드를 적용해줘. (o)

- 모호한 요청 피하기

로그인 기능을 수정해줘. (x)

.auth.py에서 JWT 토큰 만료 시간을
1시간에서 24시간으로 늘려줘. (o)

1. 토큰 절약: 프롬프트 입력

• 영어로 말하기

영어

GPT-5.x & O1/3

GPT-4 & GPT-3.5 (legacy)

GPT-3 (legacy)

Last weekend, I visited a small neighborhood bookstore near the river. The weather was clear and slightly cool, so many people were walking outside with coffee in their hands. Inside the store, soft music was playing, and the shelves were filled with novels, travel essays, and old magazines. I spent almost an hour reading the first few pages of several books before choosing one about language learning and human memory. After paying, I sat on a bench nearby and continued reading until sunset. It was a quiet and ordinary day, but it reminded me that even simple moments can stay in our minds for a long time.

Clear

Show example

Tokens	Characters
121	612

Last weekend, I visited a small neighborhood bookstore near the river. The weather was clear and slightly cool, so many people were walking outside with coffee in their hands. Inside the store, soft music was playing, and the shelves were filled with novels, travel essays, and old magazines. I spent almost an hour reading the first few pages of several books before choosing one about language learning and human memory. After paying, I sat on a bench nearby and continued reading until sunset. It was a quiet and ordinary day, but it reminded me that even simple moments can stay in our minds for a long time.

Text

Token IDs

한글

GPT-5.x & O1/3

GPT-4 & GPT-3.5 (legacy)

GPT-3 (legacy)

지난 주말, 나는 강가 근처에 있는 작은 동네 서점을 방문했다. 날씨는 맑고 약간 선선해서 많은 사람들이 커피를 손에 들고 밖을 걸어 다니고 있었다. 서점 안에서는 잔잔한 음악이 흐르고 있었고, 책장에는 소설과 여행 에세이, 오래된 잡지들이 가득 놓여 있었다. 나는 여러 권의 책 첫 부분을 읽어 보며 거의 한 시간을 보낸 뒤, 언어 학습과 인간의 기억에 관한 책 한 권을 골랐다. 계산을 마친 뒤에는 근처 벤치에 앉아 해가 질 때까지 계속 읽었다. 조용하고 평범한 하루였지만, 단순한 순간도 오랫동안 마음속에 남을 수 있다는 것을 다시 떠올리게 해 주었다.

Clear

Show example

Tokens	Characters
181	312

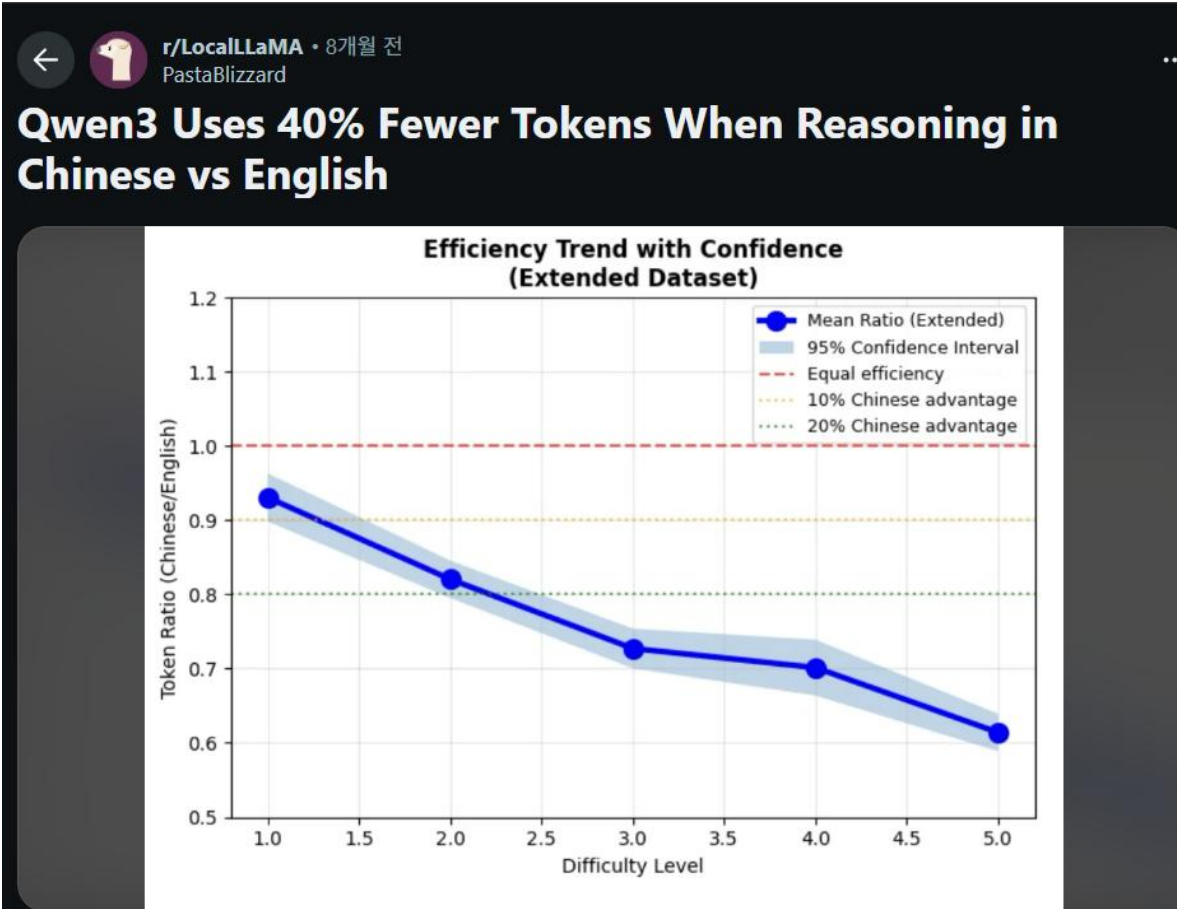
지난 주말, 나는 강가 근처에 있는 작은 동네 서점을 방문했다. 날씨는 00고 약간 선선해서 많은 사람들이 커피를 손에 들고 밖을 걸어 다니고 있었다. 서점 안에서는 00잔한 음악이 흐르고 있었고, 책장에는 소설과 여행 에세이, 오래된 잡지들이 가득 놓여 있었다. 나는 여러 권의 책 첫 부분을 읽어 보며 거의 한 시간을 보낸 뒤, 언어 학습과 인간의 기억에 관한 책 한 권을 골00다. 계산을 마친 뒤에는 근처 00치에 00아 해가 질 때까지 계속 읽었다. 조용하고 평범한 하루였지만, 단순한 순간도 오랫동안 마음속에 남을 수 있다는 것을 다시 떠올리게 해 주었다.

Text

Token IDs

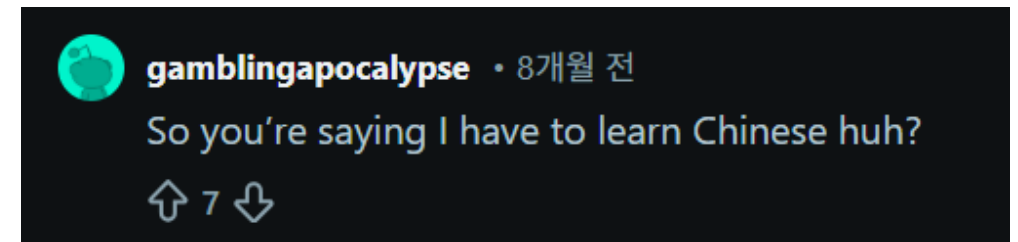
1. 토큰 절약: 프롬프트 입력

• 영어로 말하기



I tested 500 math problems from HuggingFaceH4/MATH-500 and discovered something surprising: Qwen3's Chinese Chain-of-Thought achieves 97% accuracy using only 61% of the tokens its English CoT needs. The efficiency gap grows with problem complexity - for the hardest problems (Level 5), Chinese needs just 65% of English tokens.

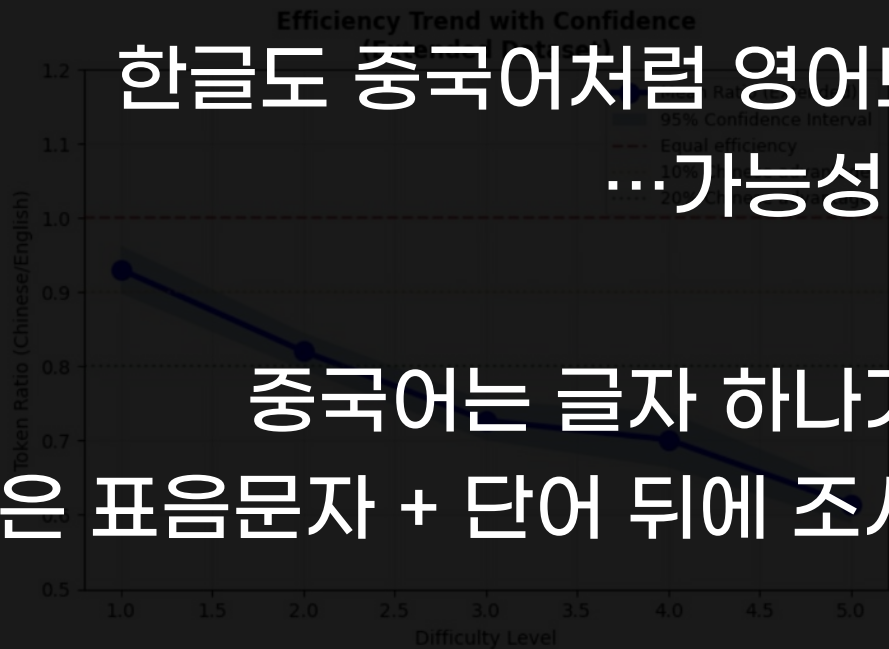
- Qwen: 중국 알리바바의 LLM
 - 중국어로 추론 시 영어보다 토큰↓
 - ‘중국어가 영어보다 저렴’



1. 토큰 절약: 프롬프트 입력

• 영어로 말하기

← r/LocalLLaMA • 8개월 전
PastaBlizzard
Qwen3 Uses 40% Fewer Tokens When Reasoning in Chinese vs English



한글도 중국어처럼 영어보다 저렴해질 날이 올까요?
...가능성은 아주 낮음

중국어는 글자 하나가 뜻을 가지는 표의문자
한글은 표음문자 + 단어 뒤에 조사가 계속 붙는 '교착어(부착어)' 특성

- Qwen: 중국 알리바바의 LLM
- 중국어로 추론 시 영어보다 토큰↓
- '중국어가 영어보다 저렴'

I tested 500 math problems from HuggingFaceH4/MATH-500 and discovered something surprising: Qwen3's Chinese Chain-of-Thought achieves 97% accuracy using only 61% of the tokens its English CoT needs. The efficiency gap grows with problem complexity - for the hardest problems (Level 5), Chinese needs just 65% of English tokens.

1. 토큰 절약: 프롬프트 입력

• 영어로 말하기

한글 토큰 효율성
 $642(2020) > 313(2022) > 181(2024)$

GPT-5.x & O1/3 GPT-4 & GPT-3.5 (legacy) GPT-3 (legacy)

지난 주말, 나는 강가 근처에 있는 작은 동네 서점을 방문했다. 날씨는 맑고 약간 선선해서 많은 사람들이 커피를 손에 들고 발을 걸어 다니고 있었다. 서점 안에서는 잔잔한 음악이 흐르고 있었고, 책장에는 소설과 여행 에세이, 오래된 잡지들이 가득 놓여 있었다. 나는 여러 권의 책 첫 부분을 읽어 보며 거의 한 시간을 보낸 뒤, 언어 학습과 인간의 기억에 관한 책 한 권을 골랐다. 계산을 마친 뒤에는 근처 벤치에 앉아 해가 질 때까지 계속 읽었다. 조용하고 평범한 하루였지만, 단순한 순간도 오랫동안 마음속에 남을 수 있다는 것을 다시 떠올리게 해 주었다.

Clear Show example

Tokens Characters
642 312

000000 000000, 000000 000000 000000000 000000 00000 000000 000000000 000000
000000. 0000000000 0000000 000000000000 000000 00000000000000 0000000000 000
000 0000000 000000 0000000 0000000000 000000000. 0000000 0000000000000 00000000 00
000000 0000000000 0000000000, 0000000000000 00000000 00000000, 0000000
000 000000000000 0000000 0000000 0000000000. 0000000 0000000 000000 000 000 0000000
00 000000 000000 000000 00 00000000 000000 000, 000000 00000000 00000000 00
0000000 000000 000 00 000000 000000000. 00000000 000000 000000000 000000 0000
00000 000000, 0000000000, 000000000000 000000000000 00000 000 00000000
0 00000 000000 000000000000 00 000000000.

Text Token IDs

GPT-5.x & O1/3 GPT-4 & GPT-3.5 (legacy) GPT-3 (legacy)

지난 주말, 나는 강가 근처에 있는 작은 동네 서점을 방문했다. 날씨는 맑고 약간 선선해서 많은 사람들이 커피를 손에 들고 발을 걸어 다니고 있었다. 서점 안에서는 잔잔한 음악이 흐르고 있었고, 책장에는 소설과 여행 에세이, 오래된 잡지들이 가득 놓여 있었다. 나는 여러 권의 책 첫 부분을 읽어 보며 거의 한 시간을 보낸 뒤, 언어 학습과 인간의 기억에 관한 책 한 권을 골랐다. 계산을 마친 뒤에는 근처 벤치에 앉아 해가 질 때까지 계속 읽었다. 조용하고 평범한 하루였지만, 단순한 순간도 오랫동안 마음속에 남을 수 있다는 것을 다시 떠올리게 해 주었다.

Clear Show example

Tokens Characters
313 312

지00 주말, 나는 00가 00000에 있는 작은 0000 서00을 방문00다. 00000는 00고 00간 0000
해서 00은 사00들이 00000를 00에 00고 00을 00어 다니고 있00다. 서00 안에서는 0000한 0000
이 00000고 있00고, 0000장에는 0000과 여000 에세이, 오래된 00지들이 가00 000어 있00다. 나
는 여러 000의 000 00 부분을 00어 보00 00의 한 시간을 보00 000, 00어 000습과 인간의 기00
에 관한 000 한 000을 0000다. 계산을 마00 000에는 00000 00치에 000아 해가 00 때00지 계
0 0000다. 조용하고 0000한 하0000지만, 00000한 00간도 오00동00 마음00에 00을 수 있다는
것을 다시 00000리게 해 주00다.

Text Token IDs

GPT-5.x & O1/3 GPT-4 & GPT-3.5 (legacy) GPT-3 (legacy)

지난 주말, 나는 강가 근처에 있는 작은 동네 서점을 방문했다. 날씨는 맑고 약간 선선해서 많은 사람들이 커피를 손에 들고 발을 걸어 다니고 있었다. 서점 안에서는 잔잔한 음악이 흐르고 있었고, 책장에는 소설과 여행 에세이, 오래된 잡지들이 가득 놓여 있었다. 나는 여러 권의 책 첫 부분을 읽어 보며 거의 한 시간을 보낸 뒤, 언어 학습과 인간의 기억에 관한 책 한 권을 골랐다. 계산을 마친 뒤에는 근처 벤치에 앉아 해가 질 때까지 계속 읽었다. 조용하고 평범한 하루였지만, 단순한 순간도 오랫동안 마음속에 남을 수 있다는 것을 다시 떠올리게 해 주었다.

Clear Show example

Tokens Characters
181 312

지난 주말, 나는 강가 근처에 있는 작은 동네 서점을 방문했다. 날씨는 00고 약간 선선해서 많은
사람들이 커피를 손에 들고 발을 걸어 다니고 있었다. 서점 안에서는 00잔한 음악이 흐르고 있었고,
책장에는 소설과 여행 에세이, 오래된 잡지들이 가득 놓여 있었다. 나는 여러 권의 책 첫 부분을 읽어
보며 거의 한 시간을 보낸 뒤, 언어 학습과 인간의 기억에 관한 책 한 권을 골00다. 계산을 마친 뒤
에는 근처 00치에 000아 해가 질 때까지 계속 읽었다. 조용하고 평범한 하루였지만, 단순한 순간도 오
랫동안 마음속에 남을 수 있다는 것을 다시 떠올리게 해 주었다.

Text Token IDs

1. 토큰 절약: 프롬프트 입력

- 프롬프트 반복
- LLM의 약점:
 - 프롬프트 정보의 순서에 민감함
 - 정보를 한 번만 보고 처리함
 - 중요한 정보가 뒤쪽에 있을 때 참조 실패 경향
 - 목록이나 구조화된 데이터에서 참조 오류 빈발

Prompt Repetition Improves Non-Reasoning LLMs

Yossi Leviathan*
Google Research
leviathan@google.com

Matan Kalman*
Google Research
matank@google.com

Yossi Matias
Google Research
yossi@google.com

Abstract

When not using reasoning, repeating the input prompt improves performance for popular models (Gemini, GPT, Claude, and Deepseek) without increasing the number of generated tokens or latency.

1 Prompt Repetition

LLMs are often trained as *causal* language models, i.e. past tokens cannot attend to future tokens. Therefore, the order of the tokens in a user's query can affect prediction performance. For example, a query of the form “<CONTEXT> <QUESTION>” often performs differently from a query of the form “<QUESTION> <CONTEXT>” (see *options-first* vs. *question-first* in Figure 1). We propose to *repeat the prompt*, i.e. transform the input from “<QUERY>” to “<QUERY><QUERY>”. This enables each prompt token to attend to every other prompt token, addressing the above. When not using reasoning, **prompt repetition improves the performance of LLMs (Figure 1) without increasing the lengths of the generated outputs or latency (Figures 2 and 3).**



1. 토큰 절약: 프롬프트 입력

- 프롬프트 반복

- *Google Research: "Prompt Repetition Improves Non-Reasoning LLMs" (2025, arXiv:2512.14982)*

- 프롬프트를 두 번 반복해서 입력하면:

- Prefill(입력 분석) 단계에서 같은 내용을 두 번 처리
- 모델 내부 표현이 더 정확해짐
- 참조 및 기억 오류가 크게 감소

- 생성 단계(출력 토큰 생성)에는 거의 영향 x

- 추론 시간 거의 증가 없음
- 출력 길이 거의 증가 없음

효과 ○	효과 미미
단순 참조 및 목록 처리	복잡한 추론 과제
구조화된 데이터 질문	이미 참조 능숙하게 처리
특정 위치의 정보 추출	-

1. 토큰 절약: 프롬프트 입력

• 프롬프트 입력 양식 추천

<General instruction>

당신은 10년 경력의 시니어 풀스택 개발자입니다.
주어진 요구사항을 바탕으로 명확하고 유지보수하기 쉬운 코드를 작성하세요.
코드 작성 후 반드시 동작 방식을 간략히 설명하세요.

</General instruction>

<Context>

- 프로젝트: 사내 직원 근태 관리 웹 애플리케이션
- 기술 스택: React (TypeScript), Node.js (Express), PostgreSQL
- 현재 개발 단계: MVP 개발 중
- 팀 규모: 프론트 2명, 백엔드 1명
- 코드 스타일: ESLint + Prettier 적용, 함수형 컴포넌트 사용

</Context>

<Instruction>

아래 요구사항을 구현하는 코드를 작성하세요.

[요구사항]

- 직원이 출근/퇴근 버튼을 누르면 현재 시각이 기록된다.
- 하루에 출근은 1회만 가능하며, 중복 클릭 시 에러 메시지를 표시한다.
- 기록된 출퇴근 시간은 화면에 표시된다.

작성 순서:

1. PostgreSQL 테이블 스키마 (attendance 테이블)
2. Node.js API 엔드포인트 (POST /attendance)
3. React 컴포넌트 (AttendanceButton.tsx)

</Instruction>

<Guideline>

- 모든 코드에 주석을 한국어로 작성하세요.
- API는 RESTful 원칙을 따르고, 에러 응답은 { status, message } 형식으로 통일하세요.
- React 컴포넌트는 useState, useEffect만 사용하고 외부 상태관리 라이브러리는 사용하지 마세요.
- 보안상 SQL Injection 방지를 위해 파라미터 바인딩을 반드시 적용하세요.
- 출력 형식:
 - 각 코드 블록 앞에 파일명 또는 용도를 명시
 - 코드 작성 후 하단에 "동작 설명" 섹션 추가 (5줄 이내)

</Guideline>

1. 토큰 절약: 프롬프트 입력

- 프롬프트 입력 양식 추천

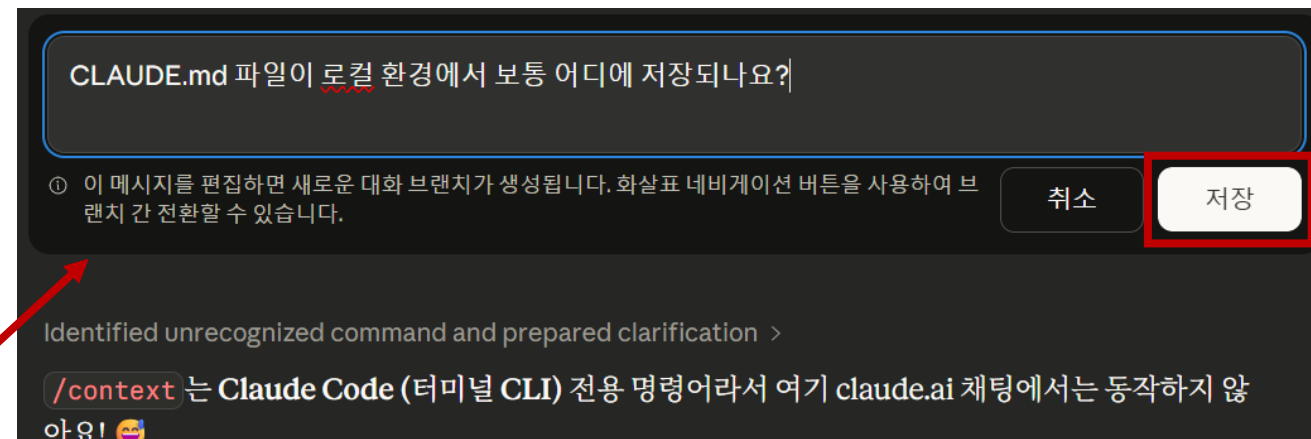
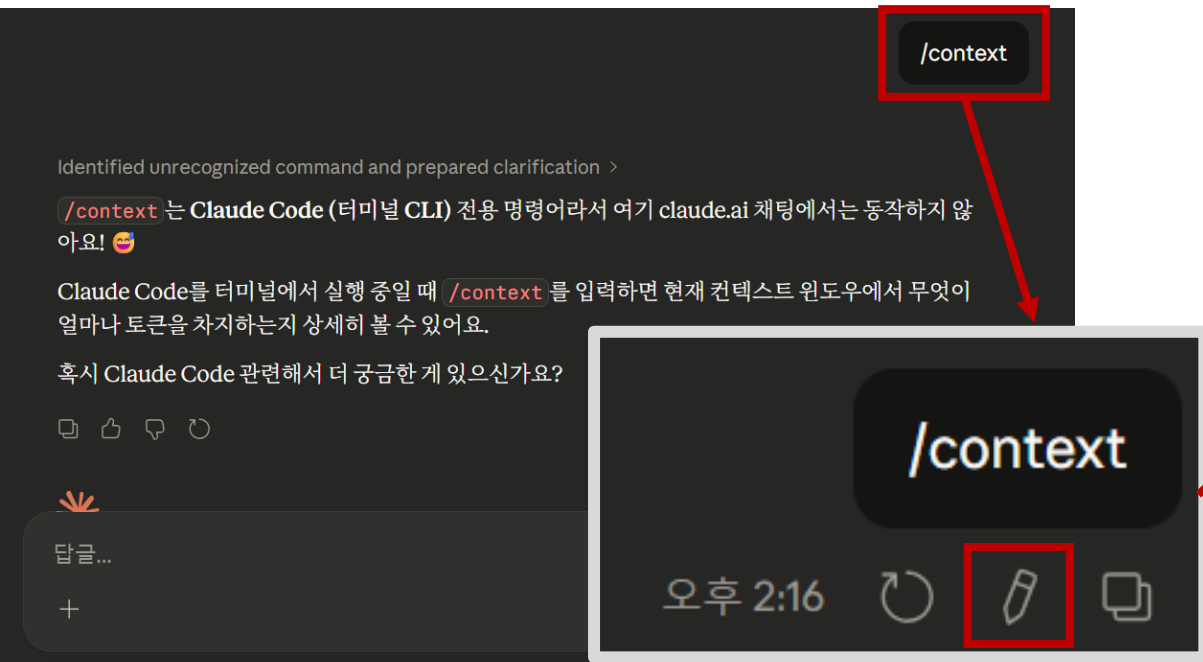
- 순서: General instruction (일반 지침), Context (맥락), Instruction (상세 지침), Guideline (가이드라인), XML 문법 사용 (혹은 마크다운 문법 사용)
 - XML e.g.,

```
<paper>{논문}</paper>
```
- 항목 사이에 적당히 줄바꿈하기
- 논문 넣을 때 레퍼런스/appendix 자르고 첨부
- 질문 한번에 여러 개 넘버링해서 주기

1. 토큰 절약: 프롬프트 입력

• 기존 입력턴 재사용

- 한 세션이 길게 이어질 때 ME1 → LLM1 → ME2 → LLM2 → ME3 → LLM3에서 ME3 → LLM3가 필요 없어졌다면 ME3를 수정하는 식으로 질문
- ⇒ ME1 ~ LLM2 맥락만 사용하게 됨



1. 토큰 절약: 세션 관리

- 세션 끊기
 - 세션을 `/compact`로 압축해도 성능이 50%~70%만 회복
→ 새 세션을 여는 게 합리적
 - 세션 닫기 전에 “다른 세션에서 계속하게 인수인계 준비”
⇒ `HANDOFF.md` 문서 생성
 - 기능 단위로 세션 구분
 - 인증 기능 완료 후 `/compact`
 - 완전히 다른 작업은 `/clear`
- +) 멀티 터미널 사용: 백엔드 / 프론트엔드 / 문서화

1. 토큰 절약: 파일 첨부

- 가급적 파일 첨부 없이, 텍스트 or 마크다운 파일 첨부
- 특히 pdf 파일
- 4,500단어 문서 $\Rightarrow$ 10만+ 토큰 소요 💀
- 마크다운 변환하면 5천 토큰 소요 $\Rightarrow$ 20배 절약
- 따로 변환 $\rightarrow$ 마크다운 파일로 전달
 - Pdftotext, 타 LLM 모델
- Or 추후 대화에도 매 턴마다 해당 토큰을 계속 로드

2. .md 활용

- .md(마크다운)이란?
 - 사람도 읽을 수 있고 AI도 읽을 수 있는 포맷
 - JSON/YAML과 달리 특정 도구에 종속되지 않음
(Claude Code, Cursor, Gemini CLI 등에서 모두 작동)
 - Git으로 버전 관리 가능
 - 순수 텍스트, BUT!! AI가 섹션 구분, 우선순위 파악, 필요한 부분만 로드 가능
- .md 문법
 - 구조화 가능

2. .md 활용: 문법

- 헤더Headers



2. .md 활용: 문법

- 블록인용BlockQuotes

```
> This is a first blockqute.  
> > This is a second blockqute.  
> > > This is a third blockqute.  
  
이 안에서는 다른 마크다운 요소를 포함할 수 있다.  
  
> ### This is a H3  
> * List  
>   ````  
>   code  
>   ````
```

This is a first blockqute.

This is a second blockqute.

This is a third blockqute.

이 안에서는 다른 마크다운 요소를 포함할 수 있다.

This is a H3

- List

```
code
```

2. .md 활용: 문법

- 목록Lists: 순서있는 / 순서없는

### 순서있는 목록 (번호)	순서있는 목록 (번호)
순서있는 목록은 숫자와 점을 사용한다.	순서있는 목록은 숫자와 점을 사용한다.
1. 첫째 2. 둘째 3. 셋째	1. 첫째 2. 둘째 3. 셋째

1. 첫째 3. 셋째 2. 둘째	1. 첫째 2. 셋째 3. 둘째

### 순서없는 목록 (글머리 기호: `*`, `+`, `-` 지원)	순서없는 목록(글머리 기호: *, +, - 지원)
혼합 사용 가능	혼합 사용 가능
* 1단계 - 2단계 + 3단계 + 4단계	• 1단계 ◦ 2단계 ▪ 3단계 ▪ 4단계

2. .md 활용: 문법

- 코드 Codes, 구분선 Divider

```
### 코드

```
````기호를 위아래에 사용 Like this.
  들여쓰기도 가능
  마치
  Like
  This
````

수평선
마크다운 문서를 미리보기로 출력할 때 *페이지 나누기* 용도로 많이 사용

* * *

- - - - -

```

### 코드

```
````기호를 위아래에 사용 Like this.
  들여쓰기도 가능
  마치
  Like
  This
```

수평선

마크다운 문서를 미리보기로 출력할 때 *페이지 나누기* 용도로 많이 사용

```
-----
-----
-----
-----
```


2. .md 활용: 문법

- 강조Emphasis

| ### 강조 | 강조 |
|--------------------------------------|-----------------------------|
| * <i>single asterisks</i> * | • <i>single asterisks</i> |
| * <u>single underscores</u> * | • <i>single underscores</i> |
| * double asterisks * | • double asterisks |
| * <u>double underscores</u> * | • double underscores |
| * cancelline * | • cancelline |

2. .md 활용

- 프로젝트 내 마크다운 파일 역할 구분

- README.md ← 사람이 읽음 “이 프로젝트가 뭔지”
- CLAUDE.md ← ClaudeCode가 읽음 “규칙과 컨벤션”
- AGENTS.md ← 모든 AI 코딩 도구가 읽음 “프로젝트를 어떻게 다룰지”
- SKILL.md ← 스킬 시스템이 읽음 “이 작업을 어떻게 수행하는지”

- 작성 원칙

- 서술형 문장보다 heading·리스트·테이블 중심으로 작성
→ 토큰 절약 + 구조 파악 용이
- 핵심만 메인 파일에 넣고, 상세 내용은 references/, examples/ 등으로 분리
→ AI가 필요할 때만 추가 파일을 로드 → 토큰 효율적 (점진적 공개)
- 스킬 파일 설계 시 단일 책임 원칙(SRP) 적용: 하나의 파일에 하나의 역할만

2. .md 활용

- 메모리 분산

- CLAUDE.md가 지나치게 길어지는 경우:
내용을 여러 보조 문서로 분리한 뒤 필요할 때만 불러오는 방식이 효과적

- 예시 구조

```
├── CLAUDE.md
└── docs
    ├── api-guide.md
    ├── db-guide.md
    └── deploy-guide.md
```

@docs/api-guide.md 참고해서 API 라우터 생성.

@docs/db-guide.md 참고해서 스키마 생성.

- SKILL.md에 핵심만 넣고, references/, assets/, examples/로 분리

- 장점

- 토큰 절약
- 유지보수 용이
- 가독성 향상

2. .md 활용

- 부분 파일 참조

- @ 기호를 사용해 파일을 참조 → 내용 전체가 컨텍스트에 포함됨
∴ 큰 파일을 무조건 통째로 읽는 방식은 비효율적
- 500줄 이하: 전체 참조 가능
500~2,000줄: 필요한 구간만 지정
2,000줄 이상: 함수나 클래스 단위로 분리

@models.py의 User 클래스만 보여줘.

@backend/ 폴더에서 jwt_token 관련 부분만 찾아 읽어줘.

- 프롬프트 캐싱

현재까지의 계획과 진행 상황을 plan-progress.md에 저장해줘.
/clear
plan-progress.md를 읽고 이어서 작업을 진행해줘.

2. .md 활용

- 부분 파일 참조

- @ 기호를 사용해 파일을 참조 **여기서 잠깐!!** 컨텍스트에 포함됨
∴ 큰 파일을 무조건 통째로 읽는 방식은 비효율적

- 500줄 이하: 전체 참조 가능
500~2,000줄: 필요한 부분만 지정
2,000줄 이상: 함수나 클래스 단위로 분리
- ???: 그래서님은 이걸 다 하고 있는 건가요...? ㅎㅎ

@models.py의 User 클래스만 보여줘.

..... 아니요.....
"auth/urls.py" 폴더에서 "jwt_token" 관련 부분만 찾아 읽어줘.

BUT!!

- 프롬프트 캐싱

현재까지의 계획과 진행 상황을 plan-progress.md에 기록해줘.
/clear
plan-progress.md를 읽고 이어서 작업을 진행해줘.

**완벽한 Claude Code 정복을 위해
열심히 공부했습니다**

같이화이팅합시다

3. LLM 툴 추천

- tokuin (Rust CLI)
 - 여러 LLM 모델의 토큰 수와 API 비용 사전 추정
 - github.com/nooscraft/tokuin
- Llmfit
 - 내 하드웨어에서 실행 가능한 로컬 LLM 및 예상 tok/s를 미리 확인
 - github.com/AlexsJones/llmfit

4. Claude Code Tips

- <https://github.com/ykdojo/claude-code-tips>
- Tip 7: 터미널 alias — 터미널 전용 단축어 개념
 - c : claude 실행
 - cc : claude --dangerously-skip-permissions 모드 실행
 - cr : 최근 대화 리스트 불러오기
 - cs : 마지막으로 종료된 세션 바로 이어서 시작하기
- Tip 0: 커스텀 상태바
 - 채팅창 하단에 현재 모델/사용량/세션 유지 시간, Context 사용량 %, 현재 브랜치 위치 표시하는 기능

4. Claude Code Tips

- <https://github.com/ykdojo/claude-code-tips>
- Tip 8: HANDOFF.md 패턴
 - 컨텍스트 소진 전 핸드오프 문서 작성 → 새 세션에서 이어받기
 - 기존에 /checkpoint 라는 장기기억 전환 기능에 통합
 - 바로 다음 세션에 이어받을 내용 + 장기기억에 답을 내용 한번에 정리함
- Tip 15-b: MCP 도구 lazy-load
 - lazy-load: MCP를 필요할 때만 호출하는 기능 ⇒ 토큰 절약

감사합니다